03-03 SigVerify Stage 구현 분석

개요

SigVerify Stage는 TPU 파이프라인의 두 번째 단계로, Fetch Stage에서 받은 트랜잭션 패킷들의 디지털 서명을 검증하는 역할을 담당합니다. 이 단계는 CPU와 GPU를 활용한 병렬 처리를 통해 높은 성능을 달성하며, 중복 제거와 배치 최적화를 통해 효율성을 극대화합니다.

SigVerifyStage 구조체 정의

```
pub struct SigVerifyStage {
thread_hdl: JoinHandle<()>,
}
```

SigVerifyStage는 단일 스레드 핸들을 관리하는 단순한 구조체입니다. 실제 검증 작업은 별도의 스레드에서 수행됩니다.

SigVerifier 트레이트

```
pub trait SigVerifier {
type SendType: std::fmt::Debug;
fn verify_batches(&self, batches: Vec<PacketBatch>, valid_packets: usize) -> Ve
fn process_received_packet(
&mut self,
```

```
_packet: &mut Packet,
_removed_before_sigverify_stage: bool,
_is_dup: bool,
) {}
fn process_excess_packet(&mut self, _packet: &Packet) {}
fn process_passed_sigverify_packet(&mut self, _packet: &Packet) {}
fn send_packets(&mut self, packet_batches: Vec<PacketBatch>) -> Result<(), Sel
}
```

SigVerifier 트레이트는 서명 검증 로직을 추상화하며, 다양한 구현체를 허용합니다.

TransactionSigVerifier 구현체

구조체 정의

```
pub struct TransactionSigVerifier {
packet_sender: BankingPacketSender,
tracer_packet_stats: SigverifyTracerPacketStats,
recycler: Recycler<TxOffset>,
recycler_out: Recycler<PinnedVec<u8>>,
reject_non_vote: bool,
}
```

주요 필드:

- **packet_sender**: 검증된 패킷을 Banking Stage로 전달하는 채널
- **tracer_packet_stats**: 추적 패킷 통계 정보
- **recycler**: 메모리 재활용을 위한 TxOffset 풀
- **recycler_out**: GPU 연산을 위한 PinnedVec 메모리 풀
- **reject_non_vote**: 투표가 아닌 트랜잭션 거부 여부

생성자 함수

```
impl TransactionSigVerifier {
pub fn new(packet_sender: BankingPacketSender) -> Self {
init(); // GPU 라이브러리 초기화
Self {
packet_sender,
tracer_packet_stats: SigverifyTracerPacketStats::default(),
recycler: Recycler::warmed(50, 4096),
recycler_out: Recycler::warmed(50, 4096),
reject_non_vote: false,
}
}

pub fn new_reject_non_vote(packet_sender: BankingPacketSender) -> Self {
let mut new_self = Self::new(packet_sender);
new_self.reject_non_vote = true;
new_self
```

```
}

}
```

서명 검증 파이프라인

1. 메인 검증 루프

```
fn verifier<const K: usize, T: SigVerifier>(

deduper: &Deduper<K, [u8]>,

recvr: &Receiver<PacketBatch>,

verifier: &mut T,

stats: &mut SigVerifierStats,

) -> Result<(), T::SendType> {

let (mut batches, num_packets, recv_duration) = streamer::recv_packet_batches(r



// 1. 랜덤 폐기 (Random Discard)

let non_discarded_packets = solana_perf::discard::discard_batches_randomly(

&mut batches,

MAX_DEDUP_BATCH,

num_packets,

);



// 2. 중복 제거 (Deduplication)
```

```
let discard_or_dedup_fail = deduper::dedup_packets_and_count_discards(
deduper,
&mut batches,
|received_packet, removed_before_sigverify_stage, is_dup| {
verifier.process_received_packet(
received_packet,
removed_before_sigverify_stage,
is_dup,
);
},
) as usize;


// 3. 초과 패킷 폐기 (Excess Packet Discard)
let mut num_packets_to_verify = num_unique;
if num_unique > MAX_SIGVERIFY_BATCH {
Self::discard_excess_packets(
&mut batches,
MAX_SIGVERIFY_BATCH,
|excess_packet| verifier.process_excess_packet(excess_packet),
);
num_packets_to_verify = MAX_SIGVERIFY_BATCH;
}
```

```
// 4. 배치 압축 (Pre-shrink)
let (pre_shrink_time_us, pre_shrink_total) = Self::maybe_shrink_batches(&mut  ba


// 5. 서명 검증 (Signature Verification)
let  mut  batches = verifier.verify_batches(batches, num_packets_to_verify);
let  num_valid_packets = count_valid_packets(
&batches,
|valid_packet| verifier.process_passed_sigverify_packet(valid_packet),
);


// 6. 배치 압축 (Post-shrink)
let (post_shrink_time_us, post_shrink_total) = Self::maybe_shrink_batches(&mut


// 7. 검증된 패킷 전달
verifier.send_packets(batches)?;


Ok(())
}
```

2. 성능 최적화 상수

```
// 50ms 목표 처리 시간 기준
const MAX_DEDUP_BATCH: usize = 165_000; // 중복 제거 최대 배치 크기
```

```
const MAX_SIGVERIFY_BATCH: usize = 5_000; // 서명 검증 최대 배치 크기

const MAX_DISCARDED_PACKET_RATE: f64 = 0.10; // 배치 압축 임계값 (10%)
```

중복 제거 메커니즘

Deduper 초기화

```
const MAX_DEDUPER_AGE: Duration = Duration::from_secs(2);

const DEDUPER_FALSE_POSITIVE_RATE: f64 = 0.001;

const DEDUPER_NUM_BITS: u64 = 63_999_979;
```

```
let mut deduper = Deduper::<2, [u8]>::new(&mut rng, DEDUPER_NUM_BITS);
```

중복 제거 과정

```
let discard_or_dedup_fail = deduper::dedup_packets_and_count_discards(

deduper,

&mut batches,

|received_packet, removed_before_sigverify_stage, is_dup| {

verifier.process_received_packet(

received_packet,

removed_before_sigverify_stage,

is_dup,
```

```
);
},
) as usize;
```

- **Bloom Filter 기반**: 확률적 데이터 구조를 사용한 효율적인 중복 검사
- **False Positive Rate**: 0.1%의 오탐률로 메모리 효율성 확보
- **주기적 리셋**: 2초마다 Deduper 리셋으로 오래된 데이터 제거

초과 패킷 처리

discard_excess_packets() 함수

```
pub fn discard_excess_packets(
batches: &mut [PacketBatch],
mut max_packets: usize,
mut process_excess_packet: impl FnMut(&Packet),
) {
// IP 주소별로 패킷 그룹화
let mut addrs = batches
.iter_mut()
.rev()
.flat_map(|batch| batch.iter_mut().rev())
.filter(|packet| !packet.meta().discard())
.map(|packet| (packet.meta().addr, packet))
```

```
.into_group_map();

// 주소별로 균등하게 max_packets 할당

while max_packets > 0 && !addrs.is_empty() {

let num_addrs = addrs.len();

addrs.retain(|_, packets| {

let cap = (max_packets + num_addrs - 1) / num_addrs;

max_packets -= packets.len().min(cap);

packets.truncate(packets.len().saturating_sub(cap));

!packets.is_empty()

});

}

// 초과 패킷 폐기

for packet in addrs.into_values().flatten() {

process_excess_packet(packet);

packet.meta_mut().set_discard(true);

}

}
```

주요 특징:

1. **공정한 분배**: IP 주소별로 균등하게 패킷 할당
2. **DoS 방어**: 단일 IP에서 과도한 패킷 전송 방지
3. **역순 처리**: 최신 패킷 우선 보존

실제 서명 검증 구현

verify_packet() 함수

```
fn verify_packet(packet: &mut Packet, reject_non_vote: bool) -> bool {
// 이미 폐기된 패킷 확인
if packet.meta().discard() {
return false;
}

let packet_offsets = get_packet_offsets(packet, 0, reject_non_vote);
let mut sig_start = packet_offsets.sig_start as usize;
let mut pubkey_start = packet_offsets.pubkey_start as usize;
let msg_start = packet_offsets.msg_start as usize;

// 서명 개수만큼 반복 검증
for _ in 0..packet_offsets.sig_len {
let pubkey_end = pubkey_start.saturating_add(size_of::<Pubkey>());
let Some(sig_end) = sig_start.checked_add(size_of::<Signature>()) else {
return false;
};
// 서명, 공개키, 메시지 추출
let Some(Ok(signature)) = packet.data(sig_start..sig_end).map(Signature::try_fro
```

```
return false;
};
let Some(pubkey) = packet.data(pubkey_start..pubkey_end) else {
return false;
};
let Some(message) = packet.data(msg_start..) else {
return false;
};
// Ed25519 서명 검증
if !signature.verify(pubkey, message) {
return false;
}
pubkey_start = pubkey_end;
sig_start = sig_end;
}
true
}
```

병렬 처리

```
lazy_static! {
static  ref PAR_THREAD_POOL: ThreadPool = rayon::ThreadPoolBuilder::new()
.num_threads(get_thread_count())
```

```
.thread_name(|i| format!("solSigVerify{i:02}"))
.build()
.unwrap();
}
```

```
pub const VERIFY_PACKET_CHUNK_SIZE: usize = 128;
```

- **Rayon 스레드 풀**: 모든 CPU 코어 활용
- **청크 단위 처리**: 128개 패킷씩 묶어서 병렬 처리
- **지연시간 최적화**: 8ms 이하의 검증 지연시간 목표

GPU 가속 지원

초기화

```
pub fn init() {
if let Some(api) = perf_libs::api() {
unsafe {
(api.ed25519_set_verbose)(true);
assert!((api.ed25519_init)(), "ed25519_init() failed");
(api.ed25519_set_verbose)(false);
}
}
}
```

GPU 메모리 관리

```
recycler: Recycler<TxOffset>, // GPU 오프셋 데이터
recycler_out: Recycler<PinnedVec<u8>>, // GPU 출력 버퍼
```

- **Pinned Memory**: GPU와 CPU 간 효율적인 데이터 전송
- **메모리 재활용**: 할당/해제 오버헤드 최소화

배치 압축 최적화

maybe_shrink_batches() 함수

```
pub fn maybe_shrink_batches(packet_batches: &mut Vec<PacketBatch>) -> (u64, us
let num_packets = count_packets_in_batches(packet_batches);
let num_discarded_packets = count_discarded_packets(packet_batches);
let discarded_packet_rate = (num_discarded_packets as f64) / (num_packets as
if discarded_packet_rate >= MAX_DISCARDED_PACKET_RATE {
shrink_batches(packet_batches);
}
// 압축 통계 반환
(shrink_time.as_us(), shrink_total)
}
```

압축 조건:

- **임계값**: 10% 이상의 패킷이 폐기된 경우
- **효과**: 메모리 사용량 감소 및 캐시 효율성 향상
- **타이밍**: 서명 검증 전후 두 번 수행

통계 및 모니터링

SigVerifierStats 구조체

```
struct SigVerifierStats {
recv_batches_us_hist: histogram::Histogram,
verify_batches_pp_us_hist: histogram::Histogram,
discard_packets_pp_us_hist: histogram::Histogram,
dedup_packets_pp_us_hist: histogram::Histogram,
batches_hist: histogram::Histogram,
packets_hist: histogram::Histogram,
num_deduper_saturations: usize,
total_batches: usize,
total_packets: usize,
total_dedup: usize,
total_excess_fail: usize,
total_valid_packets: usize,
total_shrinks: usize,
// 시간 통계들...
```

```
}
```

수집되는 메트릭:

- **처리 시간**: 수신, 검증, 폐기, 중복제거 각 단계별 시간
- **패킷 통계**: 총 패킷 수, 유효 패킷 수, 중복 패킷 수
- **배치 통계**: 배치 수, 압축 횟수
- **성능 지표**: 패킷당 처리 시간, 처리량

추적 패킷 시스템

추적 키 정의

```
const TRACER_KEY_BYTES: [u8; 32] = [
13, 37, 180, 170, 252, 137, 36, 194, 183, 143, 161, 193, 201, 207, 211, 23,
189, 93, 33, 110, 155, 90, 30, 39, 116, 115, 238, 38, 126, 21, 232, 133,
];
const TRACER_KEY: Pubkey = Pubkey::new_from_array(TRACER_KEY_BYTES);
```

추적 통계

```
pub struct SigverifyTracerPacketStats {
pub total_removed_before_sigverify_stage: usize,
pub total_tracer_packets_received_in_sigverify_stage: usize,
pub total_tracer_packets_deduped: usize,
```

```
pub total_excess_tracer_packets: usize,
pub total_tracker_packets_passed_sigverify: usize,
}
```

추적 패킷을 통해 파이프라인의 각 단계에서 패킷 처리 상황을 모니터링할 수 있습니다.

오류 처리

PacketError 열거형

```
pub enum PacketError {
InvalidLen,
InvalidPubkeyLen,
InvalidShortVec,
InvalidSignatureLen,
MismatchSignatureLen,
PayerNotWritable,
InvalidProgramIdIndex,
InvalidProgramLen,
UnsupportedVersion,
}
```

각 오류 유형에 대해 적절한 처리를 통해 시스템 안정성을 확보합니다.

요약

SigVerify Stage는 Solana TPU의 보안 관문 역할을 하며, 다음과 같은 핵심 기능을 제공합니다:

1. **고성능 서명 검증**: CPU/GPU 병렬 처리를 통한 높은 처리량
2. **효율적인 중복 제거**: Bloom Filter 기반 확률적 중복 검사
3. **공정한 자원 할당**: IP 주소별 균등한 패킷 처리
4. **적응적 배치 관리**: 동적 배치 압축을 통한 메모리 최적화
5. **실시간 모니터링**: 상세한 성능 메트릭 수집 및 보고
6. **DoS 방어**: 과도한 트래픽에 대한 보호 메커니즘

이러한 설계를 통해 SigVerify Stage는 높은 보안성과 성능을 동시에 달성하면서도 네트워크 공격에 대한 강건성을 제공합니다.